

14L1-DepthFS

Generated by Scribe.AI

December 18, 2024

Slide 1

Content

CS 24300: AI Basics

Tony Bergstrom

Slides adapted from Yexiang Xue and Tony Bergstrom

Description

This slide is a title slide for a lecture on Artificial Intelligence Basics, likely part of a course called CS 24300. It displays the course title, "CS 24300: AI Basics," and the instructor's name, "Tony Bergstrom." It also credits the source of the slides, "Slides adapted from Yexiang Xue and Tony Bergstrom," indicating that the content may be based on materials from another source. The overall presentation is simple and straightforward, focusing on the course title and instructor information. The context of the course is crucial for understanding the content of the lecture that will follow.

Figures

Slide 2

Content

Recursion

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, displays a single word: "Recursion." It's a simple title slide, likely introducing a section or topic about recursion in the context of AI algorithms or problem-solving techniques. The context of the course is crucial for understanding the specific discussion that will follow on this topic.

Figures

Slide 3

Content

A quick review of recursion

Recursion is a self reference, eventually terminating in a base case

Methods:

- A method can call itself
- Example Fibonacci

Data Structures:

- A data structure can reference itself
- Example: LinkedList is a node that points to a LinkedList
- Example: Trees are nodes that point to Trees

```
[node distance=1cm] [draw, rectangle, fill=blue!50] (f1bn) fib(n); [draw, rectangle, fill=blue!50, right of=f1bn] (f1bn1) fib(n-1); [draw, rectangle, fill=blue!50, right of=f1bn1] (f1bn2) fib(n-2); [->] (f1bn) - (f1bn1); [->] (f1bn1) - (f1bn2); [draw, rectangle, fill=blue!50, below of=f1bn, node distance=2cm] (l1nkedList) LinkedList; [draw, rectangle, fill=blue!50, left of=l1nkedList, node distance=1cm] (n1ode) Node; [->] (l1nkedList) - (n1ode); [->] (n1ode) - (l1nkedList); [draw, rectangle, fill=blue!50, below of=l1nkedList, node distance=2cm] (t1ree) Tree; [draw, rectangle, fill=blue!50, below left of=t1ree, node distance=1cm] (t1reeA) Tree A; [draw, rectangle, fill=blue!50, below right of=t1ree, node distance=1cm] (t1reeB) Tree B; [->] (t1ree) - (t1reeA); [->] (t1ree) - (t1reeB);
```

Description

This slide provides a quick review of recursion, focusing on its application in methods and data structures. It defines recursion as a self-referencing process that eventually terminates at a base case. The slide illustrates this concept with a diagram showing how a function 'fib(n)' (likely a Fibonacci sequence function) recursively calls itself with 'fib(n-1)' and 'fib(n-2)'. The diagram also demonstrates how data structures like LinkedLists and Trees can be recursively defined. A LinkedList is recursively defined as a node that points to another LinkedList. Similarly, a Tree is recursively defined as a node that points to other Trees. The visual representation using boxes and arrows clearly illustrates the recursive nature of these concepts. The context of the course (Artificial Intelligence Basics) suggests that this slide is a preliminary review of recursion, which is a fundamental concept in many AI algorithms and data structures.

Figures

Slide 4

Content

Recursive Problem Solving

Do part of the work and reduce the rest into a similar but smaller problem.

Examples:

Need to calculate $\text{Fibonacci}(n)$?

If n is 0, the answer is 1

If n is 1, the answer is 1

Otherwise, Get Help:

- Alice calculates $\text{Fibonacci}(n-1)$
- Bob calculates $\text{Fibonacci}(n-2)$
- Alice and Bob can ask others for help if they need to
- You can add Alice and Bob's answers.

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, introduces the concept of recursive problem-solving. It defines recursion as breaking down a problem into a similar but smaller subproblem. The example provided is calculating the Fibonacci sequence. The slide details the steps involved in calculating $\text{Fibonacci}(n)$: If n is 0 or 1, the answer is 1; otherwise, Alice calculates $\text{Fibonacci}(n-1)$, Bob calculates $\text{Fibonacci}(n-2)$, and the results are combined. The slide also mentions that Alice and Bob can ask others for help if needed, and that the final answer is the sum of Alice and Bob's answers. The context of the course suggests that this slide is introducing a fundamental technique in AI, where problems are often broken down into smaller, self-similar subproblems for efficient solution.

Figures

Slide 5

Content

Recursion must eventually end

The method does not always call itself

The data structure does not always link to another data structure

There are two main pieces to any recursive approach

- The Base Case - what can you do right now, in this call.
- The Recursive Case - the same problem/method/data structure, but smaller

The Recursive Case needs to be "smaller" in some sense: closer to the base case.

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, discusses the crucial requirement for recursion to terminate. It emphasizes that a recursive method or data structure does not always call itself or link to another instance of itself. The key components of a recursive approach are highlighted: the Base Case and the Recursive Case. The Base Case represents the condition that allows the recursion to stop, providing a direct solution to the problem in the current call. The Recursive Case describes how the problem is broken down into a smaller, similar subproblem, bringing the solution closer to the base case. The slide stresses that the recursive case must be "smaller" in some way, ensuring that the recursion eventually reaches the base case and terminates. The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a discussion on the design and implementation of recursive algorithms and data structures, emphasizing the importance of termination conditions for avoiding infinite loops.

Figures

Slide 6

Content

Method Recursion: a Fibonacci Example

Fibonacci is a classic recursive example

$\text{fib}(0) = 1$

$\text{fib}(1) = 1$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

The resulting call structure resembles a tree

```
[node distance=1cm] [draw, rectangle, fill=blue!50] (fib4) fib(4); [draw, rectangle, fill=blue!50, below left of=fib4] (fib3) fib(3); [draw, rectangle, fill=blue!50, below right of=fib4] (fib2) fib(2); [draw, rectangle, fill=blue!50, below left of=fib3] (fib2_1) fib(2); [draw, rectangle, fill=blue!50, below right of=fib3] (fib1) fib(1); [draw, rectangle, fill=blue!50, below left of=fib2] (fib1_1) fib(1); [draw, rectangle, fill=blue!50, below right of=fib2] (fib0) fib(0); [draw, rectangle, fill=blue!50, below left of=fib1] (fib1_2) fib(1); [draw, rectangle, fill=blue!50, below right of=fib1] (fib0_1) fib(0); [->] (fib4) -- (fib3); [->] (fib4) -- (fib2); [->] (fib3) -- (fib2_1); [->] (fib3) -- (fib1); [->] (fib2_1) -- (fib1_1); [->] (fib2_1) -- (fib0); [->] (fib1_1) -- (fib1_2); [->] (fib1_1) -- (fib0_1); [->] (fib1_2) -- (fib0_1); [->] (fib0_1) -- (fib0); [->] (fib0_1) -- (fib0);
```

Note the order in which the nodes finish execution, more on this later.

Description

This slide, part of a lecture on Artificial Intelligence Basics, illustrates method recursion using a Fibonacci example. It shows a tree diagram representing the recursive calls made to calculate $\text{Fibonacci}(4)$. The nodes in the tree represent function calls ' $\text{fib}(n)$ ' for different values of ' n ', from ' $\text{fib}(4)$ ' at the root down to ' $\text{fib}(0)$ ' at the leaves. Each node's value is calculated by recursively calling ' $\text{fib}(n-1)$ ' and ' $\text{fib}(n-2)$ ', as defined by the Fibonacci sequence formula. The diagram visually demonstrates how the recursive calls are nested and how the calculation proceeds. The slide also notes the order in which the nodes (function calls) complete their execution, a topic that will likely be discussed further in the lecture. The context of the course suggests that this slide is introducing the concept of recursion and its application in algorithms, specifically in the context of the Fibonacci sequence, which is a common example in introductory computer science and AI.

Figures

Slide 7

Content

Recursive Data Structures

Recursion is useful with some data structures like a LinkedList, Tree, or Graph.

Consider a Tree:

- Base Case: a null, or empty root node
- Recursive Case: A node with links to 1 or more subtrees

Processing the tree recursively:

- If the root node is null, done
- Otherwise, process the root node and children

Activities:

- Printing all elements
- Summing elements
- Finding a specific element
- Identifying a "best" element

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, discusses the use of recursion with specific data structures, focusing on trees. It states that recursion is useful with data structures like linked lists, trees, and graphs. The slide then specifically considers trees, outlining the base case (a null or empty root node) and the recursive case (a node with links to one or more subtrees). The slide also details the recursive processing of a tree: if the root node is null, the process is complete; otherwise, the root node and its children are processed. Finally, the slide lists common activities that can be performed recursively on a tree, such as printing all elements, summing elements, finding a specific element, and identifying a "best" element. The context of the course suggests that this slide is introducing the concept of recursion and its application in traversing and manipulating tree-based data structures, which are frequently used in AI algorithms and data representation.

Figures

Slide 8

Content

Structural Recursion: Traversal

Breadth First visits a level at a time:

- 36, 10, 78, 3, 21, 55, 105, ...
- Check the current node, add children to a queue

Depth First traverses down a branch:

- 36, 10, 3, 1, 6, 21, 15, 28, ...
- Check current node, then left, then right (other orders are possible)

[level distance=1.5cm, every node/.style=draw, rectangle, minimum size=1em, fill=blue!50] 36 child node 10 child node 3 child node 1 child node 6 child node 21 child node 15 child node 28 child node 78 child node 55 child node 45 child node 66 child node 105 child node 91 child node 120;

Note: a depth first traversal is like the call order of our Fibonacci example.

Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses structural recursion, specifically focusing on tree traversal methods. It presents a tree diagram with nodes labeled by numbers. The slide outlines two primary traversal strategies: Breadth-First and Depth-First. Breadth-First traversal visits nodes level by level, starting from the root node (36) and proceeding to the next level (10, 78), and so on. Depth-First traversal, on the other hand, explores a branch of the tree as deeply as possible before backtracking. The example shows a Depth-First traversal, where the algorithm would visit nodes in the order 36, 10, 3, 1, 6, 21, 15, 28, ... The slide also highlights that other orders are possible within the Depth-First approach, depending on the specific implementation (e.g., left-then-right, right-then-left). The diagram visually represents the tree structure and the order in which nodes would be visited using each traversal method. The note at the bottom connects the concept of Depth-First traversal to the call order of a recursive function, such as the Fibonacci sequence example, which was likely discussed in previous slides. The context of the course suggests that this slide is introducing the fundamental concepts of tree traversal algorithms and their relationship to recursive procedures.

Figures

Slide 9

Content

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, demonstrates structural recursion using a binary search tree. The slide title, "Structural Recursion: a Binary Tree Example," clearly indicates the topic. A binary search tree is depicted visually using a tree diagram. Each node in the tree is represented by a rectangle filled with a light blue color. The numbers inside the rectangles represent the values associated with each node. The tree structure is hierarchical, with the root node at the top (36) and child nodes branching downwards. The slide also includes a bulleted list describing the properties of a binary search tree: each node has a value and two children (left and right); the left child's value is less than the parent's value; and the right child's value is greater than the parent's value. These properties are crucial for the efficient search and retrieval of data within the tree structure. The context of the course suggests that this slide is introducing the concept of structural recursion in the context of tree traversal and manipulation, which are fundamental operations in many AI algorithms.

Figures

Slide 10

Content

Description

This slide, part of a lecture on Artificial Intelligence Basics, demonstrates structural recursion using a binary search tree. The title, "Structural Recursion: a Binary Tree Example," clearly indicates the topic. A binary search tree is visually depicted. Each node in the tree is represented by a rectangle filled with a light blue color. The numbers inside the rectangles represent the values associated with each node. The tree structure is hierarchical, with the root node at the top (36) and child nodes branching downwards. The slide also includes a recursive algorithm description for searching for a value X within the tree. The algorithm first examines the current node's value (V). If V equals X , the search is successful (return True). If there are no children, the value X is not present (return False). If X is less than V , the search continues recursively in the left subtree; otherwise, it continues in the right subtree. The context of the course suggests that this slide is introducing the concept of structural recursion in the context of tree traversal and manipulation, which are fundamental operations in many AI algorithms. The algorithm presented is a common way to search a binary search tree, which is an efficient data structure for storing and retrieving data.

Figures

Slide 11

Content

Description

This slide, part of a lecture on Artificial Intelligence Basics, illustrates how methods and structural recursion work together in a search algorithm, likely for a game like guessing a number between 0 and 100. A tree structure is shown, visually representing the search process. The root node is labeled "50". From this node, two branches extend downwards. The left branch leads to node "75", and the right branch leads to node "63". From "63", a branch leads to "57". Dotted lines represent the expansion of the search tree, indicating that the algorithm explores possible states by examining the current state and expanding to new states. The text emphasizes that this process of examining the current state and expanding to new states is a common technique for building a search tree during the search process. The context of the course suggests that the slide is demonstrating a recursive search algorithm, where the algorithm recursively explores possible states until the target is found. The numbers in the nodes likely represent the values being considered in the search.

Figures

Slide 12

Content

Problem Solving as Search

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, presents a simple title: "Problem Solving as Search". No further details or visual aids are present. The context suggests that the following slides will delve into the concept of representing and solving problems using search algorithms in the field of AI. The date and page number (4/5/202, 3) indicate the slide's position within a larger presentation.

Figures

Slide 13

Content

“...an agent with several immediate options of unknown value can decide what to do by first examining different possible sequences of actions that lead to states of known value, and then choosing the best sequence.”
Russell & Norving, p. 60

Description

This slide presents a quote describing a problem-solving strategy for an agent in an Artificial Intelligence context. The quote states that an agent with multiple, uncertain options can determine the best course of action by first exploring different sequences of actions that lead to states with known values. The agent then selects the sequence that yields the best outcome. The citation "Russell Norving, p. 60" indicates the source of the quote, likely a textbook or reference material on AI. The context of the course (Artificial Intelligence Basics) suggests that this slide introduces the concept of decision-making under uncertainty and the use of search algorithms to find optimal solutions.

Figures

Slide 14

Content

Problem Solving as Search

Search: Exploration of possibilities

- Abstraction of general problem solving method

Basic ingredients:

- States: describe configuration of environment
- Actions: activities which move you from one state to another
- Search algorithm: determines which actions should be tried in which order to find solution

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces the concept of problem-solving as a search process. It defines "Search" as the exploration of possible solutions. The slide further emphasizes that this approach is a general method for solving problems. The slide then lists the fundamental components of a search-based problem-solving approach:

- **States:** These represent configurations of the environment being considered. For example, in a game, a state might describe the positions of pieces on the board.
- **Actions:** These are the possible steps or moves that can be taken to transition from one state to another. In a game, actions might be "move pawn forward," "attack opponent's piece," etc.
- **Search algorithm:** This is the method used to systematically explore the possible states and actions to find a solution. The algorithm determines the order in which actions are tried.

The slide is a high-level overview, setting the stage for more detailed discussions on specific search algorithms and their applications in AI.

Figures

Slide 15

Content

Searching States – Chess is a classic example

When game-playing AI plays chess; it creates and models as many states as it can.

- At a basic level – what does the board look like if that piece is moved?
- Evaluate if it's good or bad
- Create and consider as many as possible

Some Observations:

- The search tree is large: White has 20 possible first moves, as does black. 400 states in 2 moves.
- Black and White are searching for different outcomes
- For most computers, searching the entire space isn't feasible

Before we get further into game playing, let's consider some simpler search spaces.

<https://www.bewitched.com/chess.html>

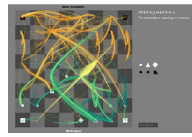
Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses the complexity of searching possible states in a game-playing AI context, using chess as an example. It highlights that when an AI plays chess, it explores and models many possible states of the game. The slide emphasizes the following points:

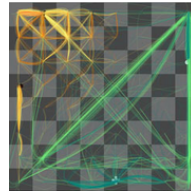
- **State Exploration:** At a basic level, the AI considers how the board changes after each move.
- **Evaluation:** The AI evaluates if a move is beneficial (good) or detrimental (bad).
- **Exhaustive Search:** The AI aims to create and consider as many possible states as possible.
- **Search Tree Size:** The slide points out the significant size of the search tree in chess. White has 20 possible first moves, and Black has 20 possible first moves, leading to 400 possible states after two moves.
- **Different Outcomes:** Black and White are searching for different outcomes in the game.
- **Computational Feasibility:** The slide emphasizes that searching the entire state space of chess is computationally infeasible for most computers.

The slide concludes by suggesting that simpler search spaces should be considered before tackling the full complexity of chess. The URL at the bottom of the slide likely links to a chess-playing resource or demonstration. The context of the course is crucial for understanding the discussion of search algorithms and their limitations in complex problem domains like game playing.

Figures



(a)



(b)

Slide 16

Content

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces the concept of maze-solving as a search problem. The slide title, "Mazes – A first problem," indicates the focus of the discussion. The slide presents a simple maze represented as a grid. The grid has rows and columns labeled 0 to 3. The walls of the maze are drawn as lines within the grid. The slide poses three key questions related to representing and solving the maze using search algorithms:

- **What is a "State"?:** This question prompts consideration of how the maze's configuration is represented as a state in the search process. A state might represent the current position of a hypothetical agent navigating the maze.
- **What "Actions" can we take:** This question focuses on the possible moves or actions an agent can perform to change its state within the maze. Possible actions might include moving up, down, left, or right.
- **How will we search the states?:** This question anticipates the need for a search algorithm to explore the possible states and actions to find a path through the maze.

The context of the course is crucial for understanding the discussion of search algorithms and their application to solving problems like mazes. The maze diagram is a visual aid to illustrate the problem domain.

Figures

Slide 17

Content

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces maze-solving as a search problem. The title, "Mazes – A first problem," indicates the focus. The slide presents a simple maze represented as a 4x4 grid. The grid lines and walls are drawn to show the maze structure. The slide defines the following key elements:

- **State:** The current location of an agent navigating the maze, represented by coordinates (x, y) .
- **Action:** Possible moves (Left, Right, Up, Down) are defined by the vector additions: $(-1, 0)$, $(1, 0)$, $(0, -1)$, $(0, 1)$. These vectors represent the change in x and y coordinates for each move.
- **Search Algorithm:** The slide poses the question of how to search the maze to find a solution.

The slide concludes by stating that the maze-solving process can be viewed as a tree-like structure. The context of the course is crucial for understanding the introduction of search algorithms and their application to solving problems like mazes. The maze diagram is a visual aid to illustrate the problem domain.

Figures

Slide 18

Content

Mazes – A first problem

Consider the first state to be location (x, y)

Move L $(-1, 0)$, R $(1, 0)$, U $(0, -1)$, or D $(0, 1)$

That's just a tree of possibilities!

... Though some may be redundant

[scale=0.7] [draw, fill=blue!50, rounded corners] (start) at (2, 3) (x, y) ; [draw, fill=blue!50, rounded corners] (L) at (1, 2) $(x - 1, y)$; [draw, fill=blue!50, rounded corners] (R) at (3, 2) $(x + 1, y)$; [draw, fill=blue!50, rounded corners] (U) at (2, 4) $(x, y - 1)$; [draw, fill=blue!50, rounded corners] (D) at (2, 2) $(x, y + 1)$;

[->] (start) - (L) node[midway, left] L; [->] (start) - (R) node[midway, right] R; [->] (start) - (U) node[midway, above] U; [->] (start) - (D) node[midway, below] D;

[draw, fill=blue!50, rounded corners] (LL) at (0.5, 1) $(x - 2, y)$; [draw, fill=blue!50, rounded corners] (LR) at (3.5, 1) $(x + 2, y)$; [draw, fill=blue!50, rounded corners] (LU) at (1.5, 5) $(x - 1, y - 1)$; [draw, fill=blue!50, rounded corners] (LD) at (1.5, 0) $(x - 1, y + 1)$; [draw, fill=blue!50, rounded corners] (RU) at (4.5, 5) $(x + 1, y - 1)$; [draw, fill=blue!50, rounded corners] (RD) at (4.5, 0) $(x + 1, y + 1)$;

[->] (L) - (LL) node[midway, left] L; [->] (L) - (LU) node[midway, left] U; [->] (L) - (LD) node[midway, left] D; [->] (R) - (LR) node[midway, right] R; [->] (R) - (RU) node[midway, right] U; [->] (R) - (RD) node[midway, right] D; [->] (U) - (LU) node[midway, above] U; [->] (U) - (RU) node[midway, above] R; [->] (U) - (UD) node[midway, above] U; [->] (D) - (LD) node[midway, below] L; [->] (D) - (RD) node[midway, below] R; [->] (D) - (DD) node[midway, below] D;

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces maze-solving as a search problem. The title "Mazes – A first problem" clearly indicates the topic. The slide visually represents a simple maze-solving scenario using a tree structure. The root node of the tree is labeled (x, y) , representing the starting location. The branches emanating from the root node represent possible moves: Left (L), Right (R), Up (U), and Down (D). Each move corresponds to a change in the x and y coordinates, as specified by the vectors $(-1, 0)$, $(1, 0)$, $(0, -1)$, and $(0, 1)$, respectively. The slide highlights that this branching structure represents a tree of possible states or locations that an agent might explore while navigating the maze. The notation $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, and $(x, y+1)$ are used to label the child nodes, indicating the new position after each move. The slide also mentions that some paths might be redundant, implying that the search algorithm needs to consider efficiency in exploring the possible states. The context of the course is crucial for understanding the introduction of search algorithms and their application to solving problems like mazes. The tree diagram is a visual aid to illustrate the problem domain and the possible states that can be explored.

Figures

Slide 19

Content

Mazes – A first problem

For our maze

- Start at (0,0), Search for (2,3)
- Move by expanding the state L,R,U,D as possible
- The smaller we make the tree – the faster we find a solution
- Identify redundant states
- Don't explore redundant states (don't get into loops)

This removes whole branches of our tree that are less useful:

- (0,0), (1,0), (0,0)
- (0,0), (1,0), (1,1), (1,0)
- (0,0), (1,0), (1,1), (1,0), (0,0)
- Etc.

Traverse the tree to find the solution:

Note, a Depth first search without removing redundant states would explore

LRLRLRLR... [scale=0.5] [step=1cm,gray,very thin] (0,0) grid (4,4); (0.5,0.5) – (3.5,0.5); (0.5,1.5) – (0.5,3.5); (0.5,3.5) – (3.5,3.5); (1.5,0.5) – (1.5,1.5); (2.5,0.5) – (2.5,1.5); (2.5,1.5) – (3.5,1.5); (1.5,2.5) – (3.5,2.5); (1.5,3.5) – (2.5,3.5); (2.5,3.5) – (3.5,3.5); (2.5,2.5) – (3.5,2.5); (2.5,2.5) – (3.5,2.5); (3.5,2.5) – (3.5,3.5); (0,0) node[below left] 0; (1,0) node[below] 1; (2,0) node[below] 2; (3,0) node[below] 3; (0,1) node[left] 0; (0,2) node[left] 1; (0,3) node[left] 2; (0,4) node[left] 3;

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces maze-solving as a search problem. The title "Mazes – A first problem" clearly indicates the topic. A 4x4 grid represents a maze with walls. The slide describes a strategy for solving the maze using a search algorithm. The algorithm starts at the top-left corner (0,0) and aims to reach the bottom-right corner (2,3). The algorithm expands possible states (moves) in a tree-like structure, exploring possible paths. The key idea is to identify and avoid redundant states (loops) to make the search more efficient. The slide emphasizes that the smaller the search tree, the faster a solution is found. The example shows how redundant states (e.g., (0,0), (1,0), (0,0)) can be avoided. The slide also mentions Depth-First Search (DFS) as a search algorithm that would explore all possible paths without removing redundant states, leading to a potentially very large search space. The context of the course is crucial for understanding the introduction of search algorithms and their application to solving problems like mazes. The maze diagram is a visual aid to illustrate the problem domain and the possible states that can be explored.

Figures

Slide 20

Content

Mazes – A first problem

Each node is a position in the maze

Each connection represents moving L, R, D, or U in the maze, without redundancies

Just like "Guess a Number", we can generate this tree as we go

- The tree structure represents paths to all locations in the maze
- Explore all possible next moves
- If there is no next move (deadend), backtrack up the tree to an unexplored path

```
[scale=0.6] [step=1cm,gray,very thin] (0,0) grid (4,4); (0.5,0.5) – (3.5,0.5); (0.5,1.5) – (0.5,3.5); (0.5,3.5) – (3.5,3.5); (1.5,0.5) – (1.5,1.5); (2.5,0.5) – (2.5,1.5); (2.5,1.5) – (3.5,1.5); (1.5,2.5) – (3.5,2.5); (1.5,3.5) – (2.5,3.5); (2.5,3.5) – (3.5,3.5); (2.5,2.5) – (3.5,2.5); (0,0) node[below left] 0; (1,0) node[below] 1; (2,0) node[below] 2; (3,0) node[below] 3; (0,1) node[left] 0; (0,2) node[left] 1; (0,3) node[left] 2; (0,4) node[left] 3; [draw, circle, fill=blue!20, inner sep=1pt] at (0,0) 0,0; [draw, circle, fill=blue!20, inner sep=1pt] at (0,1) 0,1; [draw, circle, fill=blue!20, inner sep=1pt] at (0,2) 0,2; [draw, circle, fill=blue!20, inner sep=1pt] at (0,3) 0,3; [draw, circle, fill=blue!20, inner sep=1pt] at (1,0) 1,0; [draw, circle, fill=blue!20, inner sep=1pt] at (1,1) 1,1; [draw, circle, fill=blue!20, inner sep=1pt] at (1,2) 1,2; [draw, circle, fill=blue!20, inner sep=1pt] at (1,3) 1,3; [draw, circle, fill=blue!20, inner sep=1pt] at (2,0) 2,0; [draw, circle, fill=blue!20, inner sep=1pt] at (2,1) 2,1; [draw, circle, fill=blue!20, inner sep=1pt] at (2,2) 2,2; [draw, circle, fill=blue!20, inner sep=1pt] at (2,3) 2,3; [draw, circle, fill=blue!20, inner sep=1pt] at (3,0) 3,0; [draw, circle, fill=blue!20, inner sep=1pt] at (3,1) 3,1; [draw, circle, fill=blue!20, inner sep=1pt] at (3,2) 3,2; [draw, circle, fill=blue!20, inner sep=1pt] at (3,3) 3,3;
```

Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces maze-solving as a search problem. The title "Mazes – A first problem" clearly indicates the topic. A 4x4 grid represents a maze with walls. The slide describes a strategy for solving the maze using a tree-like structure. Each node in the tree represents a position (x, y) in the maze. Connections between nodes represent possible moves (Left, Right, Up, Down). The slide emphasizes that the tree structure represents all possible paths through the maze. The algorithm explores possible next moves, and if a dead end is reached, it backtracks to an unexplored path. The slide uses the analogy of "Guess a Number" to illustrate the tree-building process. The context of the course is crucial for understanding the introduction of search algorithms and their application to solving problems like mazes. The maze diagram and the tree diagram are visual aids to illustrate the problem domain and the possible states that can be explored.

Figures

Slide 21

Content

Revisiting Fibonacci

Many recursive approaches benefit from removing redundant states.

Build your approach to avoid doing work

2 or more times if once will suffice

Why calculate fib(2), two times?

```
[level distance=1.5cm, every node/.style=rectangle, rounded corners, draw, minimum size=1cm, align=center]
fib(4) child node fib(3) child node fib(2) child[missing] child node fib(1) child node fib(1) child node fib(0)
; [below of=fib(3), xshift=-1cm] fib(2); [below of=fib(3), xshift=1cm] fib(1); [below of=fib(2), xshift=-1cm]
fib(1); [below of=fib(2), xshift=1cm] fib(0);
```

Description

This slide, part of a lecture on Artificial Intelligence Basics, revisits the Fibonacci sequence problem, focusing on the efficiency of recursive solutions. The title "Revisiting Fibonacci" indicates a review of a previously discussed topic. The slide highlights that many recursive approaches can be improved by removing redundant calculations. The text "Build your approach to avoid doing work 2 or more times if once will suffice" emphasizes the need for memoization or dynamic programming to avoid recalculating the same Fibonacci numbers multiple times. The diagram shows a tree structure representing the recursive calls for calculating fib(4). The key point is that fib(2) is calculated twice in the recursive process, demonstrating the redundancy that can be eliminated. The context of the course is crucial for understanding the concept of dynamic programming and its application to optimizing recursive algorithms. The tree diagram visually illustrates the recursive calls and the repeated calculations of fib(2).

Figures

Slide 22

Content

Count the number of calls to fib(n)

n →	0	1	2	3	4	5	6	7	8	9
fib(n)	1	1	2	3	5	8	13	21	34	55
Count	1	1	3	5	9	15	25	41	67	109
n →	10									
fib(n)	89									
Count	177									

$$\text{Count}(\text{fib}(n)) = \text{Count}(\text{fib}(n-1)) + \text{Count}(\text{fib}(n-2)) + 1$$

[node distance=1.5cm] [draw, rectangle, fill=teal!50, rounded corners] (fibn) fib(n); [draw, rectangle, fill=teal!50, rounded corners, below left of=fibn] (fibn1) fib(n-1); [draw, rectangle, fill=teal!50, rounded corners, below right of=fibn] (fibn2) fib(n-2); [draw, rectangle, fill=teal!50, rounded corners, right of=fibn2] (fib0) fib(0); [draw, rectangle, fill=teal!50, rounded corners, left of=fibn1] (fib1) fib(1); [->, thick, dashed] (fibn) - (fibn1); [->, thick, dashed] (fibn) - (fibn2); [->, thick, dashed] (fibn1) - (fib0); [->, thick, dashed] (fibn2) - (fib1);

Description

This slide from an Artificial Intelligence Basics course presents a table and a diagram illustrating the computational cost of calculating the Fibonacci sequence. The table shows the Fibonacci numbers (fib(n)) for values of n from 0 to 10, along with the corresponding count of recursive calls required to compute each Fibonacci number. The table demonstrates that the number of calls grows exponentially with n. The diagram visually represents the recursive calls needed to compute fib(n). Each node in the diagram represents a call to the fib function. The arrows show the recursive calls to compute fib(n-1) and fib(n-2). The formula "Count(fib(n)) = Count(fib(n-1)) + Count(fib(n-2)) + 1" is given, highlighting the recursive nature of the calculation and the constant overhead of +1 for each call. The context of the course suggests that the slide is discussing the efficiency of recursive algorithms and the potential for optimization techniques like dynamic programming to improve performance.

Figures

Slide 23

Content

Lets optimize a bit

[node distance=1.5cm] [draw, rectangle, fill=teal!50, rounded corners] (fib1) fib(1); [draw, rectangle, fill=teal!50, rounded corners, right of=fib1] (fib0) fib(0); [draw, rectangle, fill=teal!50, rounded corners, below left of=fib1] (fib2) fib(2); [draw, rectangle, fill=teal!50, rounded corners, below of=fib0] (fib1b) fib(1); [draw, rectangle, fill=teal!50, rounded corners, below of=fib2] (fib3) fib(3); [draw, rectangle, fill=teal!50, rounded corners, right of=fib0] (fib4) fib(4); [draw, rectangle, fill=teal!50, rounded corners, right of=fib4] (fib3b) fib(3); [draw, rectangle, fill=teal!50, rounded corners, above right of=fib3b] (fib5) fib(5); [draw, rectangle, fill=teal!50, rounded corners, below right of=fib3] (fib2b) fib(2); [draw, rectangle, fill=teal!50, rounded corners, below of=fib4] (fib4b) fib(4); [->, thick] (fib1) -- (fib2); [->, thick] (fib0) -- (fib2); [->, thick] (fib2) -- (fib3); [->, thick] (fib1b)

	List Cache	
	0	1
	1	1
– (fib3); [->, thick] (fib4) – (fib5); [->, thick] (fib3b) – (fib5); [->, thick] (fib2b) – (fib4b);	2	2
	3	3
	4	5
	5	8

Description

This slide, part of a lecture on Artificial Intelligence Basics, presents an optimization strategy for calculating Fibonacci numbers. The slide shows a directed acyclic graph (DAG) representing the recursive calls needed to compute Fibonacci numbers up to fib(5). Each node in the graph represents a call to the fib function, and the arrows indicate the dependencies between the calls. For example, the node ‘fib(5)’ depends on the nodes ‘fib(4)’ and ‘fib(3)’. The graph visually demonstrates the redundant calculations that occur in a naive recursive implementation. A "List Cache" table is included, showing the Fibonacci numbers from fib(0) to fib(5) and their corresponding values. The context of the course suggests that the slide is introducing dynamic programming as a technique to optimize recursive algorithms by storing and reusing previously computed results, thereby avoiding redundant calculations. The slide is preparing the audience for a discussion on memoization or tabulation techniques to improve the efficiency of recursive Fibonacci calculations.

Figures

Slide 24

Content

Comparison

[title=Comparison, xlabel=n, ylabel=Value, legend pos=south east, ymode=log, xmin=0, xmax=50, ymin=1, ymax=2e8, xtick=0,1,2,3,4,5,6,7,8,9,10,20,30,40,50, ytick=1,10,100,1000,10000,100000,1000000,10000000,100000000,2e8, log basis y=10,] [blue, mark=*, mark size=1.5pt] coordinates (0, 1) (1, 1) (2, 2) (3, 3) (4, 5) (5, 8) (6, 13) (7, 21) (8, 34) (9, 55) (10, 89) (20, 6765) (30, 832040) (40, 102334155) (50, 20365011074); fib(n); [green, mark=*, mark size=1.5pt] coordinates (0, 1) (1, 1) (2, 3) (3, 5) (4, 9) (5, 15) (6, 25) (7, 41) (8, 67) (9, 109) (10, 177) (20, 6765) (30, 832040) (40, 102334155) (50, 20365011074); C-fib; [green!50!black, mark=*, mark size=1.5pt] coordinates (0, 1) (1, 1) (2, 3) (3, 5) (4, 7) (5, 9) (6, 11) (7, 13) (8, 15) (9, 17) (10, 19) (20, 6765) (30, 832040) (40, 102334155) (50, 20365011074); C-fibE;

n →	0	1	2	3	4	5	6	7	8	9
fib(n)	1	1	2	3	5	8	13	21	34	55
C-fib	1	1	3	5	9	15	25	41	67	109
C-fibE	1	1	3	5	7	9	11	13	15	17
n →	10									
fib(n)	89									
C-fib	177									
C-fibE	19									
n →	50									
fib(n)	20,365,011,074									
C-fib	40,730,022,147									
C-fibE	99									

Description

This slide from an Artificial Intelligence Basics course presents a comparison of different methods for calculating Fibonacci numbers. A graph displays the growth of the Fibonacci sequence ($\text{fib}(n)$) and two optimized versions (C-fib and C-fibE) as a function of n . The y-axis is logarithmic, highlighting the exponential growth of the Fibonacci sequence. The x-axis represents the input value n . The graph shows that the optimized methods (C-fib and C-fibE) have significantly lower computational costs compared to the standard recursive approach. The table below the graph provides a numerical comparison of the different methods for various values of n . The table shows the Fibonacci numbers ($\text{fib}(n)$) and the number of function calls required for each method (C-fib and C-fibE). The context of the course suggests that the slide is discussing the efficiency of different algorithms for calculating Fibonacci numbers, emphasizing the importance of optimization techniques in AI applications where computational speed is critical. The table shows that the C-fibE method has a much lower computational cost than the C-fib method, which in turn has a lower cost than the standard recursive approach. The difference in computational cost becomes increasingly significant as n increases.

Figures

Slide 25

Content

Searching Graphs and Maps

Description

This slide, likely the title slide for a section of a lecture on Artificial Intelligence Basics, introduces the topic of searching graphs and maps. It's a straightforward title slide, indicating the subject matter of the upcoming content.

Figures

Slide 26

Content

Searching in more complicated structures

Our search approach is built on a tree

But what if our problem is built on a graph?

What if there are more than one ways to reach a state?

If a solution is found, will it be the best?

Description

This slide, likely part of a lecture on Artificial Intelligence Basics, introduces the topic of searching in more complex structures than trees. It poses a series of questions to transition the discussion from tree-based search algorithms to graph-based search algorithms. The questions highlight the need for more sophisticated search strategies when dealing with graphs, where multiple paths to a solution might exist. The questions specifically address: 1) the difference between tree and graph search, 2) the possibility of multiple solutions, and 3) the optimality of the solution found. The context suggests that the lecture will cover graph search algorithms and their properties, such as exploring all possible paths and finding the best solution (e.g., shortest path, lowest cost). The slide is a prelude to a discussion on graph traversal algorithms like Breadth-First Search (BFS) or Depth-First Search (DFS), and potentially more advanced algorithms like A* search.

Figures

Slide 27

Content

Example Map

[circle, fill=green!50, minimum size=5pt, inner sep=0pt] (start) at (2, 7) ; [below of=start, xshift=0.5cm]
Start; [circle, fill=red!50, minimum size=5pt, inner sep=0pt] (goal) at (6, 5) ; [below of=goal, xshift=0.5cm]
Goal;